MODULE 4 · LECTURE 19

CS 6530 Applied Cryptography

Key-management foundations
Symmetric-key distribution
Key hierarchies

Module 4 – Key Management, Certificates and PKI

FROM CRYPTOGRAPHIC OPERATIONS TO KEY MANAGEMENT

We Already Use Many Keys — But They Don't All Live the Same Life

PERSISTENT / LONGER-LIVED

Provisioned or retained across operations

- **Signing / identity private key**
  ECDSA · Ed25519
- **TLS server authentication private key**
  paired with certificate
- **CA signing private key**
  trust anchor infrastructure
- **Pre-shared key (PSK)**
  retained shared secret

EPHEMERAL / SESSION-ORIENTED

Generated or derived for a limited interaction

Ephemeral ECDHE key pair
↓
ECDHE shared secret
↓
HKDF
↓
Handshake / traffic secrets
↓
Traffic keys

authenticates *derives session keys*

Server auth key + cert → **TLS HANDSHAKE** ← ECDHE → secret → HKDF → traffic keys

Persistent and ephemeral key material coexist in the same TLS handshake.

? THINK LIKE AN ENGINEER

Which of these keys existed before this TLS connection began?

INSIGHT

Different roles → different lifetimes → different management requirements.

Standards anchors: NIST SP 800-57 Part 1 · RFC 8446

A key's role and intended lifetime determine how it must be managed.

KEY ROLE × DATA STATE × TIME

Key Lifetime Meets Data Lifetime

DATA AT REST

Database / storage / archive

Stored Data

encrypted with

DEK — Data Encryption Key

protected / wrapped under

KEK — Key Encryption Key

anchored in KMS / HSM

Persistent; rotated per policy or cryptoperiod.

DATA IN MOTION

Client ⇄ secure connection ⇄ Server

Auth key + Cert (server)

Ephemeral ECDHE

↓

shared secret

↓

HKDF

↓

traffic keys

Traffic keys: short-lived. Auth: may persist.

DATA IN USE

Application → cryptographic operation

Application

↓

encrypt / decrypt / sign

Protected module (HSM / KMS)

↓

Result

Key stays protected; not exposed as raw material.

Related — not identical

Data lifetime

Session lifetime

Key cryptoperiod



THINK LIKE AN ENGINEER

Should a key remain active for as long as the data it protects must exist?

INSIGHT

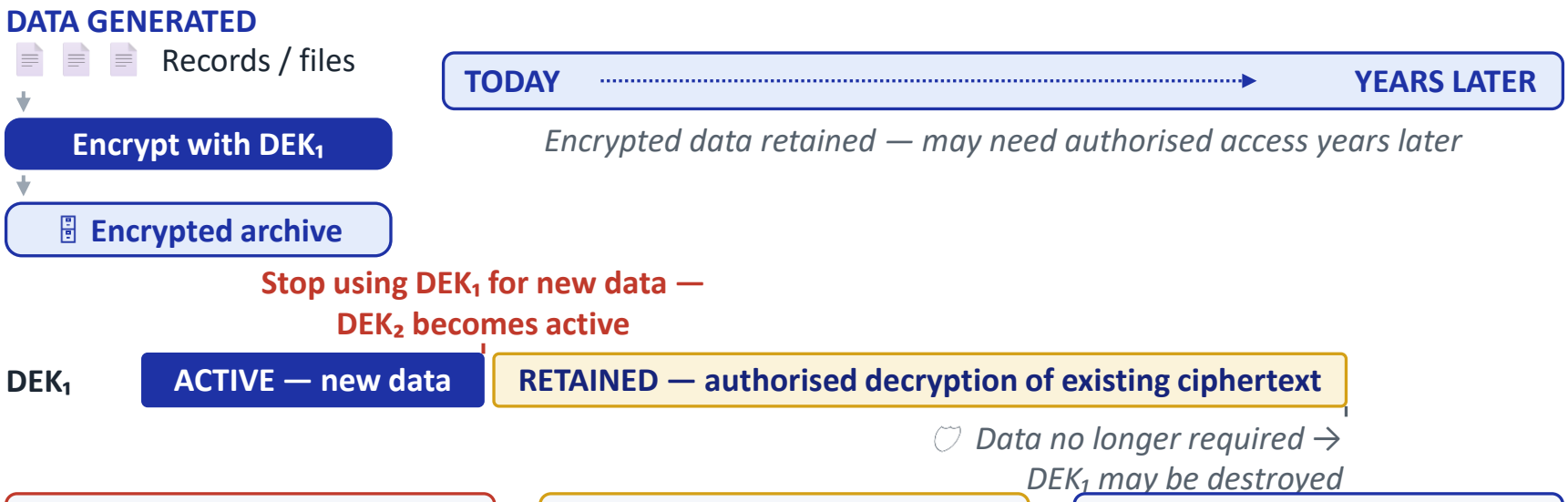
Related — but need not be equal.

Standards anchor: NIST SP 800-57 Part 1

Key lifetime follows purpose, policy and risk—not data state alone.

CRYPTOPERIOD IS NOT THE SAME AS DATA RETENTION PERIOD

Data May Outlive a Key's Active Use



DEK$_1$ no longer encrypts new data. Destroy it?
Not necessarily.

Re-encrypt all old ciphertext immediately?
Not necessarily.

Can DEK$_1$ keep encrypting new data indefinitely?
No.

CRYPTOPERIOD
Time span a key is authorised for a specific use.

A key may stop being authorised for new encryption while remaining securely retained for authorised decryption of previously protected data.

? THINK LIKE AN ENGINEER
If a key is no longer active but cannot yet be destroyed, what happens to it next?

Standards anchor: NIST SP 800-57 Part 1

Data retention can extend beyond a key's active cryptoperiod.

STANDARDS-BASED KEY LIFECYCLE

A Cryptographic Key Has a Lifecycle

⚠ *Compromise → stop authorised use → replace / revoke → assess exposure*

PRE-OPERATIONAL

Prepare for use

- Generate
- Register
- Distribute

OPERATIONAL

Authorised for use

ACTIVE

🔑 DEK_1 → 🔒 protected data

Continues until replacement is required.

REPLACEMENT / ROTATION
DEK_1 → post-operational
DEK_2 → new lifecycle

POST-OPERATIONAL

Not for new use

🗄 DEK_1 retained

- Deactivate / expire
- Retain if required
- Authorised decrypt

Still needed for old data.

Exception: revoked

DESTROYED

Not required

- 🔑
- Secure erase
- Ø destroyed

Not recoverable.

PROTECTION THROUGHOUT THE LIFECYCLE
Access control | Key protection | Audit & accountability | Backup/recovery where required | Policy

? THINK LIKE AN ENGINEER

Does every key pass through every lifecycle stage?

INSIGHT

The lifecycle framework is common; the path and duration depend on the key's role.

Standards anchor: NIST SP 800-57 Part 1

Keys move from creation/establishment through authorised use, retirement and eventual destruction.

ONE FRAMEWORK — DIFFERENT LIFECYCLE PROFILES

Different Keys, Different Lifecycles

Conceptual lifetimes — not to scale

📜 **Persistent identity / CA signing key**

Generate securely → provision / certify → protected operational use → rollover / replacement → retain (post-operational, if required) → destroy

Persistent • High-value • Strong lifecycle controls

🔒 **Database data encryption key (DEK)**

Generate → activate → DEK_1 active (encrypt new data) → DEK_1 retained / DEK_2 active for new data → destroy when no longer required

Protected data may outlive DEK_1's active encryption period.

♻️ **TLS ephemeral / traffic key material**

Generate / derive → use → update / retire → erase

ephemeral ECDHE • traffic secrets / keys

Session / connection-oriented • Protocol managed

Notice the bars above: persistent → medium-duration → session-only.
Do not read exact durations from them.

? THINK LIKE AN ENGINEER

Which lifecycle controls disappear when a key lives for only one session?

INSIGHT

Role + Risk + Policy + Protocol → Lifecycle requirements.

Standards anchor: NIST SP 800-57 Part 1

Short-lived keys need fewer persistent controls—not fewer security controls.

FROM KEY LIFETIME TO KEY CONTROL

What Must Key Management Control?

CRYPTOGRAPHIC KEY

Authorised • Protected • Accountable

1 · WHO?

- Owner
- Admin
- App

2 · WHERE?

- Software
- TPM / HSM
- KMS

3 · WHEN?

- Activate
- Use period
- Replace
- Retain
- Destroy

4 · FOR WHAT?

What is the key authorised to do?

- Encrypt
- Sign
- Agree
- Wrap key

Same algorithm, different purpose.

5 · WHAT IF?

What if the key is lost or compromised?

LOST / UNAVAILABLE
Backup/recovery • availability impact

COMPROMISED
Stop use • revoke • assess exposure

INSIGHT

Key management controls cryptographic key material, its authorised use, its protection and its lifecycle.

THINK LIKE AN ENGINEER — ? Alice —————— Bob

Alice and Bob need shared symmetric key material. Must they always run a key-agreement protocol to obtain it?

Standards anchors: NIST SP 800-57 Part 1 · NIST SP 800-130

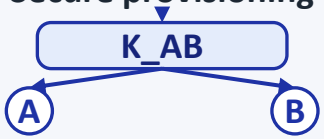
Managing a key means controlling who can use it, where, when, for what purpose—and what happens when things go wrong.

SYMMETRIC KEY DISTRIBUTION

How Can Shared Symmetric Key Material Be Obtained?

1 · PRE-PROVISION

Secure provisioning



Shared secret installed before it is needed
PSK, device or admin provisioning

Trust begins here.

2 · TRANSPORT / DISTRIBUTE

Trusted source / distributor

Protected delivery



Securely conveyed to the parties that need it

Example pattern: KDC

3 · AGREE / DERIVE



authenticated ECDH exchange

Shared secret

KDF

Symmetric keys

Established cryptographically.

ECDH = key agreement

STANDARDS VOCABULARY
*Key establishment includes mechanisms such as **key agreement** and **key transport**; systems may also begin with pre-provisioned key material.*

LONGER-LIVED / PERSISTENT

SHORT-LIVED / SESSION-ORIENTED

PSK / persistent relationship keys — often provisioned, managed or distributed; session / traffic keys — often dynamically established or derived.

Lifetime does not uniquely determine how a key was obtained.

? THINK LIKE AN ENGINEER
Does an ephemeral key have to be established by Diffie–Hellman?

Standards anchors: ISO/IEC 11770 family · NIST SP 800-57 Part 1

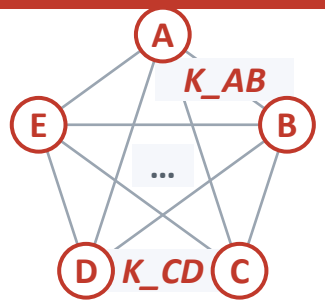
Shared key material may be provisioned, transported/distributed, agreed or derived—the mechanism depends on purpose and trust.

THE KEY-DISTRIBUTION SCALING PROBLEM

Why Not Give Every Pair a Permanent Shared Key?

CAN WE REDUCE THE PERSISTENT KEY RELATIONSHIPS?

PAIRWISE LONG-TERM SECRETS

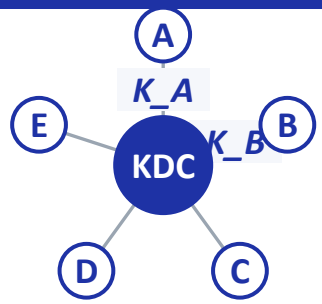


n(n−1)/2 persistent keys

10 entities → 45 keys

Every pair needs its own permanent secret.

KDC MODEL



≈ n persistent relationships

Session keys created on demand

Reduces persistent pairwise relationships—not total session keys ever generated.

PAIRWISE MESH → TRUSTED DISTRIBUTOR

SCALING
n(n−1)/2 → n persistent relationships

TRUST
KDC becomes security-critical

? THINK LIKE AN ENGINEER

If Alice and Bob share no permanent key, how can the KDC securely give both the same session key?

INSIGHT

Centralized distribution trades pairwise-key scaling for concentrated trust.

Standards anchor: ISO/IEC 11770-2

A KDC replaces many permanent pairwise secrets with persistent relationships to trusted infrastructure.

TRUSTED SYMMETRIC-KEY DISTRIBUTION

KDC: Persistent Trust Establishes a Short-Lived Session Key



Alice and Bob do not share a permanent K_AB.

1 · REQUEST — *Alice → KDC: "I need a session key for Bob."*

2 · CREATE — *KDC generates a fresh session key* ***K_AB***

3 · PROTECT FOR EACH PARTY

{K_AB}_KA	{K_AB}_KB
Protected via Alice–KDC relationship	*Protected via Bob–KDC relationship*

Exact delivery sequence is protocol-dependent.

K_AB

4 · SESSION — short-lived session key for Alice and Bob

REAL-WORLD: KERBEROS

KDC: AS · TGS

↓

TGT

↓

Service Ticket

↓

Client → Server

AS = Authentication Service · TGS = Ticket-Granting Service · TGT = Ticket-Granting Ticket · Service Ticket = access to the target service

Kerberos adds tickets, authentication and freshness mechanisms around the KDC model.

INSIGHT

Persistent trust with the KDC enables short-lived pairwise session keys on demand.

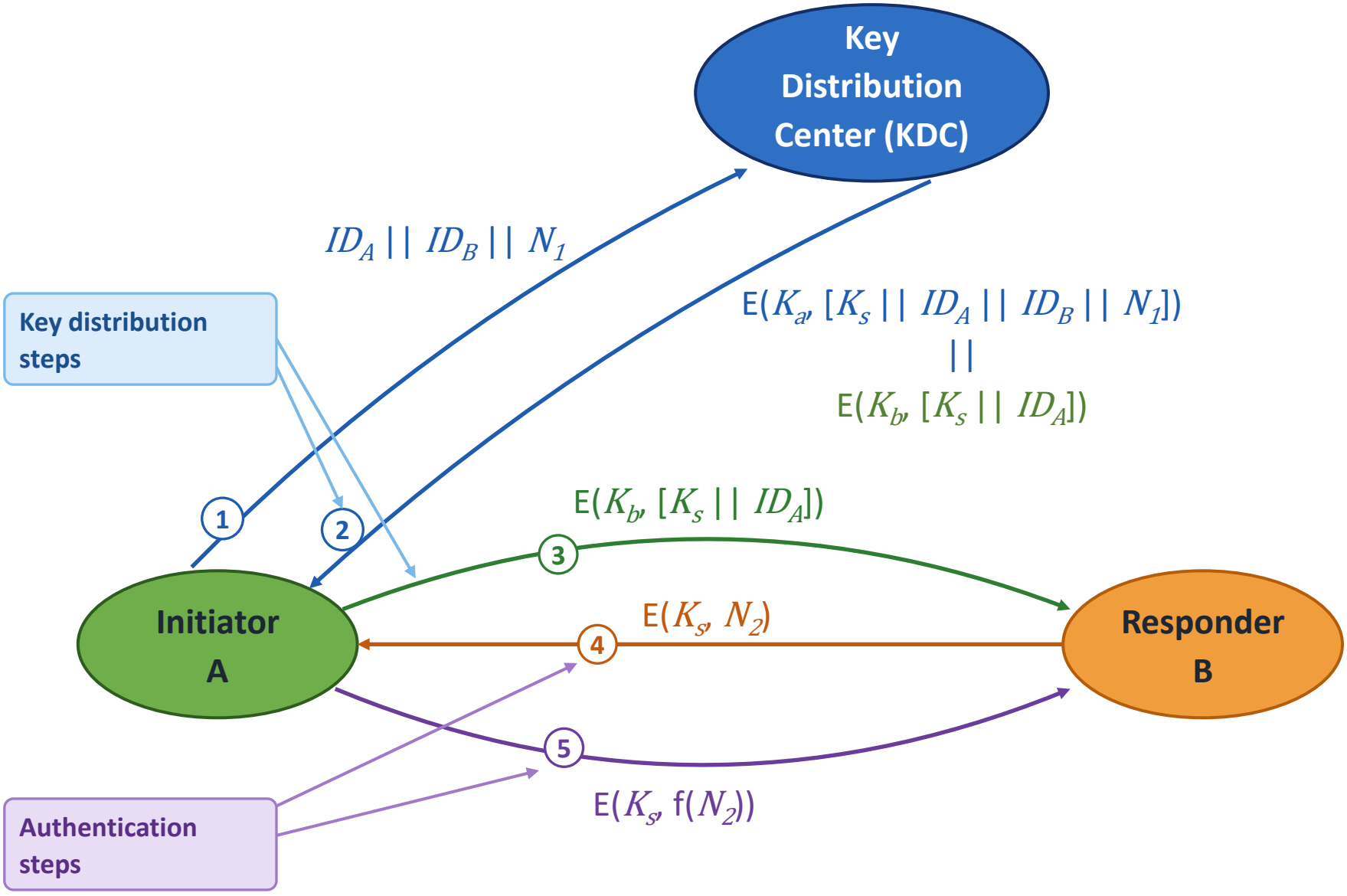
? THINK LIKE AN ENGINEER

Why use a persistent key to protect another key instead of using it directly for all data?

Standards anchor: ISO/IEC 11770-2 · Applied example: Kerberos

A KDC uses persistent trust relationships to securely establish short-lived keys for communicating parties.

KDC Key Distribution: A Concrete Message Flow



SYMMETRIC KEY DISTRIBUTION — CONCRETE FLOW

KDC Key Distribution: A Concrete Message Flow



Alice and Bob have no permanent pairwise key.

1 · REQUEST — *Alice → KDC: {ID_A, ID_B, N_1}* *(N_1 = freshness / request binding)*

2 · CREATE — *KDC generates a fresh session key* ***K_s***

ALICE'S PROTECTED PACKAGE — protected under K_A
contains: K_s · ID_B · N_1 · Bob's sealed package →
FOR BOB: E_KB(K_s, ID_A)

3 · FORWARD — Alice → Bob: the sealed package for Bob

Protected with K_B

Alice can carry it — but cannot open it. Bob opens it using K_B.

Alice ═══ *K_s* ═══ *Bob* — *SESSION KEY DISTRIBUTED*

AUTHENTICATION + FRESHNESS CONFIRMATION
4 · *B → A: E_Ks(N_2)* — *fresh challenge* **5** · *A → B: E_Ks(f(N_2))* — *proves possession of K_s*
Fresh challenge-response helps defend against replay.

? THINK LIKE AN ENGINEER

Why should the KDC generate a fresh (K_s) for Alice and Bob instead of giving them a long-term shared key (K_AB)?

INSIGHT

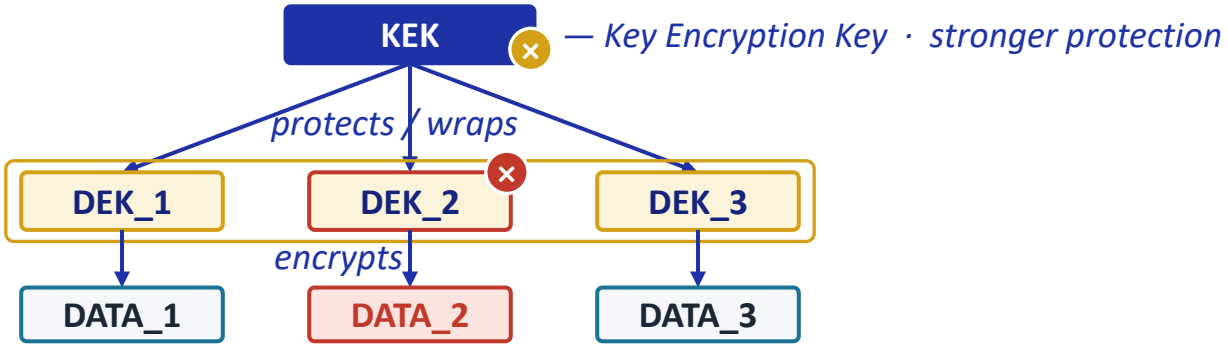
Short-lived K_s limits exposure of one session; persistent K_A or K_B compromise has broader consequences.

Standards anchor: ISO/IEC 11770-2

Persistent trust relationships can securely distribute fresh session keys, with authentication and freshness protecting the resulting communication.

KEY HIERARCHIES

DEK, KEK and the Key Hierarchy



DEK = Data Encryption Key — KEK and DEK are roles, not different key types.

CONTAINMENT — *compromise of one independently managed DEK need not expose data protected by other DEKs.*

BLAST RADIUS — *a higher-level key may place multiple protected keys at risk—so it warrants stronger protection.*

HIERARCHY IS NOT LIFETIME ORDERING

"Higher" means what a key protects — not how long it must live.

No universal ordering of KEK/DEK lifetimes — it depends on role, policy, risk and design.

BEFORE
KEK_1 wraps DEK · DEK encrypts DATA
$C_K = Wrap(KEK_1, DEK)$

REWRAP

AFTER — KEK REPLACED
KEK_2 wraps the SAME DEK
$C_K' = Wrap(KEK_2, DEK)$

BULK CIPHERTEXT UNCHANGED — *the DEK is rewrapped; the bulk data need not be re-encrypted.*

? THINK LIKE AN ENGINEER

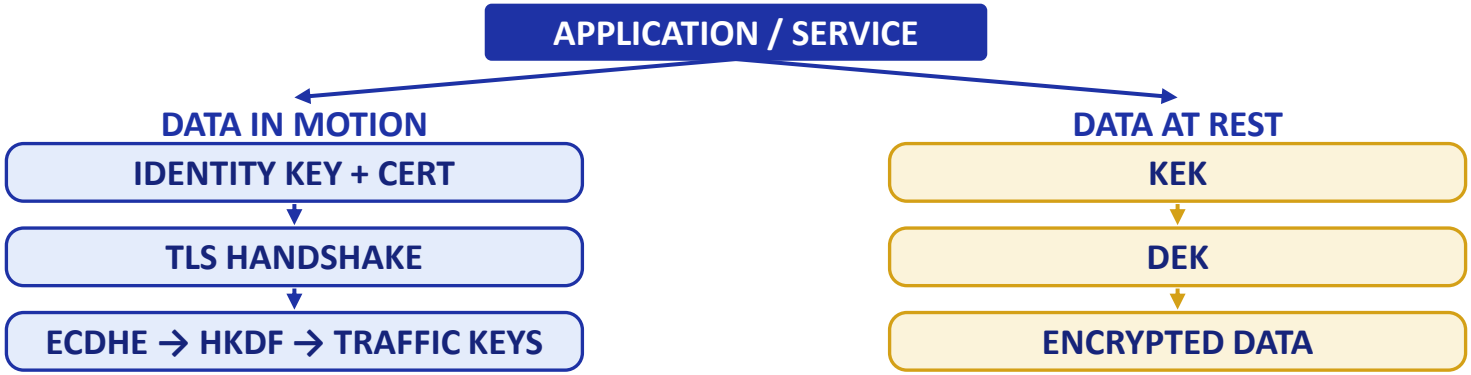
Does a key higher in the hierarchy necessarily have a longer cryptoperiod?

Standards anchor: NIST SP 800-57 Part 1 · RFC 3394 · RFC 5649

Key hierarchies separate roles and help contain compromise; hierarchy does not prescribe key lifetime.

PUTTING KEY MANAGEMENT TOGETHER

One System, Many Keys, Many Lifetimes



Persistent authentication + ephemeral key agreement coexist.

KEK and DEK have independent, managed cryptoperiods.

Data lifetime, DEK cryptoperiod and KEK cryptoperiod are not the same.

No universal lifetime ordering — as with any key hierarchy.

KEY-MANAGEMENT CONTROLS (apply across roles and throughout their lifecycles)
Authorization · Least privilege · Separation of duties · Audit & accountability · Rotation · Recovery · Secure destruction

THERE IS NO SINGLE "SYSTEM KEY"

A real system combines keys with different roles, lifetimes, dependencies and protection requirements.

? THINK LIKE AN ENGINEER

Are "DEK" and "KEK" different kinds of keys—or roles that keys perform in a system?

Standards anchors: NIST SP 800-57 Part 1 · RFC 8446 · RFC 3394/5649

Key management coordinates many keys across their roles, lifecycles and trust relationships.

TAKEAWAYS

Key Management — What Should We Remember?

1 · ROLE DRIVES MANAGEMENT

ROLE → LIFECYCLE → CONTROLS

Different key roles require different protection and usage controls

2 · LIFETIME, CRYPTOPERIOD, HIERARCHY

Data lifetime ≠ key cryptoperiod

Hierarchy ≠ lifetime ordering

Not the same: lifetime, cryptoperiod, hierarchy.

3 · MANAGE THE WHOLE LIFECYCLE

PRE-OP → OPERATIONAL → POST-OP → DESTROYED

Controls apply throughout the lifecycle.

4 · SHARED KEYS MUST COME FROM SOMEWHERE

PROVISION · TRANSPORT · AGREE/DERIVE

Alice — KDC — Bob → fresh session key

A KDC replaces pairwise secrets with trust, distributing fresh keys on demand.

5 · KEY HIERARCHIES SEPARATE ROLES

KEK → protects/wraps → DEK → encrypts → DATA

Some keys protect data; other keys protect keys.

Independent DEKs → compartmentalization.

KEY MANAGEMENT

Determines which key may be used, by whom, for what purpose, for how long — and how it is protected throughout its life.

? THINK LIKE AN ENGINEER

THINK — a public key need not be secret — but how do we know whose public key it is?

Cryptography creates and establishes keys; key management governs their safe use throughout their lifecycle.

REFERENCES & FURTHER READING

Key Management — Standards and Directed Reading

NIST SP 800-57 Part 1 Rev. 5 — PRIMARY ANCHOR

Recommendation for Key Management: Part 1 — General

For: key types & usage, cryptoperiods, lifecycle, protection and compromise considerations.

SYSTEM DESIGN & GENERATION

NIST SP 800-130 — *CKMS design & functions.*

NIST SP 800-152 — *key generation guidance.*

KEY ESTABLISHMENT & SYMMETRIC DISTRIBUTION

ISO/IEC 11770 — *key establishment.*

ISO/IEC 11770-2 — *symmetric-key establishment/distribution mechanisms*

KEY WRAPPING

RFC 3394 — *AES Key Wrap Algorithm.*

RFC 5649 — *AES Key Wrap with Padding.*

PROTOCOL CONTEXT

RFC 8446 — *TLS 1.3 auth + ephemeral keys.*

RFC 4120 — *Kerberos KDC/ticket auth.*

FIPS 140-3 — *crypto module security.*

TEXTBOOK READING FOR THIS LECTURE

Paar & Pelzl — Understanding Cryptography

CH. 14 — Key Establishment

Broader treatment of key establishment concepts.

Stallings — Cryptography and Network Security

CH. 15, §§15.1-15.2 — Symmetric Key Distribution

Concrete symmetric-key/KDC treatment used in this lecture.

Start with NIST SP 800-57 Part 1 for the standards view; use Paar Ch. 14 and Stallings Ch. 15 for textbook treatment.